



**ESCOLA DO
FUTURO**

CI/CD com GitHub Actions

Oficina 2 — Deploy Contínuo e Estratégias Avançadas

Público: Iniciante / Intermediário · **Carga horária:** 4 horas (Oficina 2 de 2)

Robson Barbosa Souza

Escola do Futuro de Goiás - Paulo Renato de Souza

Agenda da Oficina 2

Bloco 1 — Ambientes e Deploy (1h)

1. Do CI ao CD: a ponte
2. Ambientes de deploy
3. Environments e aprovações

Bloco 2 — Estratégias de release (1h)

4. Recreate, rolling, blue-green e canary
5. Rollback e versionamento

Bloco 3 — Workflows avançados (1h)

6. Reutilização, composite actions e concorrência

Bloco 4 — Segurança, observabilidade e prática (1h)

7. Segurança do pipeline · Métricas DORA · Estudo de caso

Recap: o que vimos na Oficina 1

- CI integra e valida o código continuamente
- GitHub Actions = workflows YAML + eventos + runners
- Jobs, steps, secrets, artefatos e cache
- Boas práticas e proteção de branches

 Agora vamos **entregar**: do código verde ao deploy em produção.

Do CI ao CD: a ponte

CI responde

O código está **integrado e validado**?

CD responde

O código pode ser **entregue com segurança**?



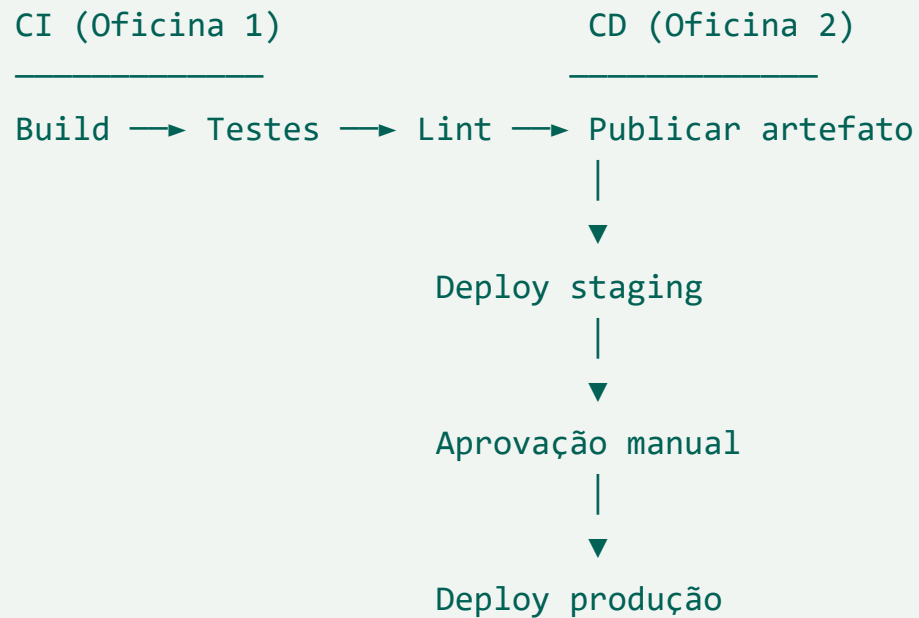
O CI termina onde o CD começa: no artefato pronto para deploy.

O que é Deploy Contínuo?

CD (Continuous Delivery/Deployment): automatizar a liberação do software para ambientes, de forma **repetível, auditável e segura**.

- Entrega contínua: deploy automático até staging, aprovação para produção
- Deploy contínuo: produção totalmente automatizada
- O artefato da CI é o **insumo** do CD

0 pipeline completo



 CI valida, CD entrega.

Bloco 1

Ambientes e Deploy

Ambientes de deploy

Ambiente	Objetivo
development	Testes locais e integração
staging	Validação próxima de produção
production	Usuários finais

- Cada ambiente tem **configuração própria**
- O código avança em **etapas** (promotion)
- Falhas são capturadas **antes** de produção

Por que separar ambientes?

- Reduz o risco de quebrar produção
- Permite validar com dados realistas (staging)
- Isola configurações e credenciais
- Cria **rastreabilidade** do deploy

⚠ Deploy direto em produção sem staging é a receita do caos.

Environments no GitHub

Recurso do GitHub Actions que agrupa **regras, secrets e variáveis** por ambiente.

- `development`, `staging`, `production`
- Criados em **Settings** → **Environments**
- Cada um com secrets e aprovações próprias

💡 Environments dão **governança** ao deploy.

Environments na prática

```
jobs:
  deploy-staging:
    environment: staging
    runs-on: ubuntu-latest
    steps:
      - run: ./deploy.sh staging

  deploy-prod:
    needs: deploy-staging
    environment: production
    runs-on: ubuntu-latest
    steps:
      - run: ./deploy.sh production
```



O job herda os secrets e regras do environment.

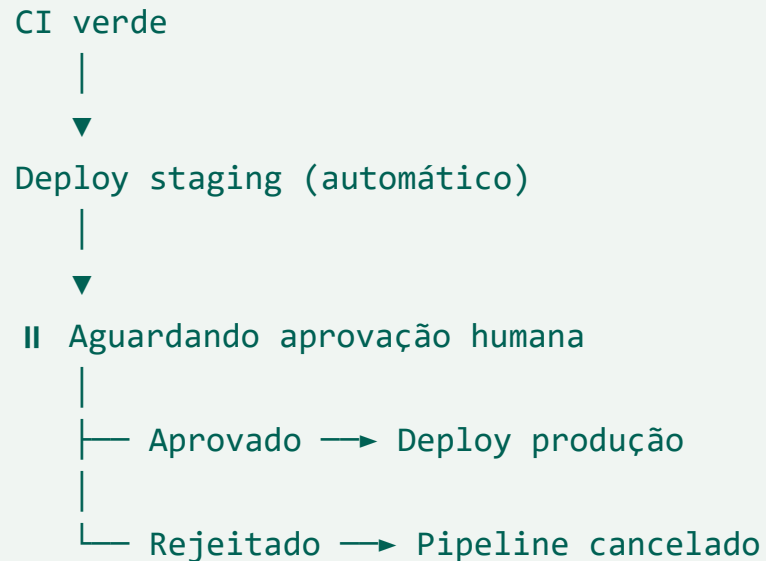
Regras de proteção de ambientes

Settings → Environments → production

- **Required reviewers:** aprovação humana obrigatória
- **Wait timer:** espera configurável antes do deploy
- **Deployment branches:** restringe branches permitidos

 Produção protegida: ninguém deploys sozinho.

Aprovação manual na prática



💡 O deploy fica **pausado** até um reviewer aprovar.

Secrets por ambiente

```
jobs:
  deploy-prod:
    environment: production
    steps:
      - run: ./deploy.sh
        env:
          API_TOKEN: ${ secrets.API_TOKEN }
```

- O mesmo nome pode ter **valores diferentes** por ambiente
- Staging usa credenciais de staging; produção, as reais
- Secrets de produção **nunca** vazam para outros ambientes

Variáveis por ambiente

```
jobs:
  deploy:
    environment: production
    steps:
      - run: echo "URL: ${vars.APP_URL}"
```

- `vars.` para configuração não sensível
- `secrets.` para credenciais
- Ambos podem ser definidos **por environment**

 Mesmo workflow, comportamentos diferentes por ambiente.

Bloco 2

Estratégias de Release e Rollback

Estratégias de release

Como a nova versão **substitui** a versão atual em produção.

- **Recreate**: para e sobe
- **Rolling**: substitui gradualmente
- **Blue-green**: troca entre ambientes
- **Canary**: libera para uma fração

 A escolha depende de risco, custo e tolerância a downtime.

Recreate (deploy direto)

Antes: [v1]



Durante: [] ← aplicação fora do ar



Depois: [v2]

Características

- Simples e barato
 - Downtime total
 - Rollback lento



Aceitável para ambientes internos e janelas de manutenção.

Rolling deploy

```
[ v1 ][ v1 ][ v1 ][ v1 ]  
  ▼  
[ v2 ][ v1 ][ v1 ][ v1 ]  
  ▼  
[ v2 ][ v2 ][ v1 ][ v1 ]  
  ▼  
[ v2 ][ v2 ][ v2 ][ v2 ]
```

Características

- Sem downtime
- Progressivo e controlável
 - Versões convivem durante o deploy

Blue-green

Blue (v1) ← tráfego atual
Green (v2) ← deploy + testes



Troca o roteador para Green



Blue (v1) ← reserva p/ rollback
Green (v2) ← tráfego atual

Características

- Rollback instantâneo (troca de roteador)
 - Custo de manter dois ambientes

Canary deploy

```
graph TD; A[100% → v1] --> B[95% → v1 | 5% → v2 (observação)]; B --> C[50% → v1 | 50% → v2]; C --> D[100% → v2];
```

100% → v1
▼
95% → v1 | 5% → v2 (observação)
▼
50% → v1 | 50% → v2
▼
100% → v2

Características

- Risco mínimo com métricas reais
 - Exige observabilidade e automação de tráfego

Comparativo de estratégias

Estratégia	Downtime	Rollback	Complexidade
Recreate	Sim	Lento	Baixa
Rolling	Não	Médio	Média
Blue-green	Não	Instantâneo	Média
Canary	Não	Instantâneo	Alta

💡 Comece simples (recreate/rolling) e evolua conforme a maturidade.

Quando usar cada estratégia?

- **Recreate**: ferramentas internas, janelas de manutenção
- **Rolling**: serviços stateless com muitos nós
- **Blue-green**: aplicações críticas com rollback rápido
- **Canary**: alto tráfego com telemetria madura

⚠ Estrat gia sofisticada sem observabilidade   risco, n o benef cio.

Rollback: o plano B

Voltar para uma versão **anterior conhecida e saudável**.

- Reverter o deploy (re-deploy da versão anterior)
- Reverter o código (git revert) e re-executar o pipeline
- Restaurar banco de dados quando necessário

 Todo deploy deve ter um **caminho de volta**.

Rollback na prática

1. Identifique a versão saudável (tag)

```
git tag -l "v*"
```

2. Reverta o código problemático

```
git revert <hash-do-commit>
```

3. Faça push – o pipeline re-executa

```
git push origin main
```

 Automatize: um workflow de rollback disparado por tag acelera a recuperação.

Versionamento semântico (SemVer)

MAJOR.MINOR.PATCH

2.4.1

| | └─ PATCH: correção de bug (compatível)
| └─ MINOR: nova funcionalidade (compatível)
└─ MAJOR: mudança incompatível

💡 Tags + SemVer = releases rastreáveis e rollback preciso.

Tags e Releases no GitHub

Tag

- Marca um commit: `git tag v2.4.1`
- Ponto de referência para deploy e rollback

Release

- Tag + notas de versão + artefatos
- Criada pela interface ou pela API

 Deploy por release = rastreabilidade completa.

Disparando deploy por release

```
on:
  release:
    types: [published]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: ./deploy.sh ${github.ref_name }
```

💡 O deploy usa o nome da release (ex.: **v2.4.1**) como versão.

Bloco 3

Workflows Avançados

Reutilização de workflows

Evite copiar e colar pipelines entre repositórios.

- **Workflow reutilizável:** pipeline chamado por outros
- **Composite action:** agrupa steps reutilizáveis
- **Templates:** padrão da organização

 DRY também vale para CI/CD.

Definindo um workflow reutilizável

```
# .github/workflows/deploy.yml
name: Deploy

on:
  workflow_call:
    inputs:
      environment:
        type: string
        required: true
    secrets:
      DEPLOY_TOKEN:
        required: true

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - run: ./deploy.sh ${ inputs.environment }
        env:
          TOKEN: ${ secrets.DEPLOY_TOKEN }
```

Chamando o workflow reutilizável

```
# .github/workflows/ci.yml
jobs:
  deploy:
    uses: ../github/workflows/deploy.yml
    with:
      environment: staging
    secrets:
      DEPLOY_TOKEN: ${ secrets.DEPLOY_TOKEN }
```

- `uses` com caminho local ou `org/repo/.github/workflows/x.yml@ref`
- Reutilização entre repositórios da organização

Composite actions

Agrupa **steps** reutilizáveis em uma única action.

- Encapsula lógica repetida (setup, lint, deploy)
- Distribuível entre repositórios
- Versionável como qualquer action

 Ideal para padronizar passos do time.

Composite action: exemplo

```
# .github/actions/setup-project/action.yml
name: Setup Project
description: Instala e configura o projeto
runs:
  using: composite
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with:
        node-version: 20
    - run: npm ci
      shell: bash
```



Em composite actions, `run` exige `shell:` explícito.

Reusable workflow × Composite action

Aspecto	Reusable	Composite
Escopo	Workflow inteiro	Steps
Chamada	<code>uses:</code> no job	<code>uses:</code> no step
Inputs	<code>workflow_call</code>	<code>inputs</code>
Secrets	Declarados	<code>{{ inputs }}</code>
Uso ideal	Pipeline padrão	Passo padrão

💡 Reusable para pipelines; composite para passos.

Concorrência entre execuções

Evita deploys simultâneos e corridas no mesmo ambiente.

```
concurrency:  
  group: ${{ github.workflow }}-${{ github.ref }}  
  cancel-in-progress: true
```

 Push rápido no mesmo branch cancela a execução anterior.

Concurrency na prática

```
concurrency:  
  group: deploy-${{ github.ref }}  
  cancel-in-progress: false
```

- `cancel-in-progress: true` → cancela a anterior
- `false` → enfileira (útil para deploys)
- Agrupe por branch, ambiente ou release

⚠ Para deploy, prefira **enfileirar** em vez de cancelar.

Matrix avançada

```
strategy:
  matrix:
    include:
      - os: ubuntu-latest
        node: 20
      - os: windows-latest
        node: 22
```

- `include` adiciona combinações específicas
- `exclude` remove combinações
- Útil para SOs e versões críticas

💡 Use com moderação: cada combinação consome minutos.

Condições e expressões

steps:

- run: echo "Sucesso!"
if: success()
- run: echo "Falhou..."
if: failure()
- run: echo "Sempre roda"
if: always()



success(), failure(), always() e cancelled() controlam o fluxo.

Contextos e expressões

Contexto	Exemplo
github	<code>\${{ github.ref }}</code> , <code>\${{ github.event_name }}</code>
secrets	<code>\${{ secrets.TOKEN }}</code>
vars	<code>\${{ vars.APP_URL }}</code>
env	<code>\${{ env.NODE_ENV }}</code>
matrix	<code>\${{ matrix.node }}</code>
needs	<code>\${{ needs.build.outputs.ver }}</code>



Contextos conectam dados do GitHub ao workflow.

Bloco 4

Segurança do Pipeline

Princípios de segurança em CI/CD

- Menor privilégio: **permissões mínimas**
- Secrets protegidos e rotacionados
- Dependências auditadas (supply chain)
- Código de terceiros tratado como **não confiável**
- Logs sem dados sensíveis

 O pipeline é parte da superfície de ataque do projeto.

Permissões mínimas

```
permissions:  
  contents: read  
  issues: none  
  pull-requests: read
```

- O token padrão (`GITHUB_TOKEN`) tem escopo amplo
- Restrinja por workflow e por job
- Conceda `write` apenas quando necessário



`permissions:` no topo vale para todo o workflow.

Secrets avançados

- Escopo: repositório, environment ou organização
- Rotação periódica e revogação imediata
- Nunca imprimir em logs ou artefatos
- Prefira **OIDC** a chaves de longa duração

⚠ Secret vazado = credencial comprometida. Rotacione já.

Riscos: pull_request_target

Executa no contexto do **branch base**, com acesso a secrets.

- Útil para comentar PRs com segurança
- **Perigoso** se o código do PR for executado
- Nunca faça checkout e execute código do PR nesse evento

 Trate PRs de terceiros como código não confiável.

Supply chain: dependências

- **Dependabot:** alertas e PRs de atualização
- **Pinning:** use versões fixas das actions (`@v4`)
- **Lockfiles:** `package-lock.json` , `requirements.txt` com hash
- **Dependency review:** bloqueia PRs com dependências vulneráveis

💡 Sua cadeia de dependências é tão crítica quanto seu código.

OIDC: credenciais sem segredos

Autenticação federada com o provedor de nuvem.

```
permissions:  
  id-token: write  
  
steps:  
  - uses: aws-actions/configure-aws-credentials@v4  
    with:  
      role-to-assume: arn:aws:iam::123456789012:role/deploy
```

- Sem chaves de longa duração no repositório
- Token curto e vinculado ao workflow

💡 OIDC elimina a classe mais comum de vazamento de credenciais.

Auditoria e logs

- **Security log** do GitHub registra eventos de Actions
- Histórico de execuções com quem disparou e o que rodou
- Retenção e exportação de logs
- Revisão periódica de workflows e permissões



Rastreabilidade é pré-requisito de conformidade.

Bloco 5

Observabilidade e Qualidade

Observabilidade do pipeline

- Status visível: **badges**, Actions, notificações
- Tempo de cada job e step
- Taxa de falhas e reprodutibilidade
- Integração com métricas de entrega

 Se você não mede o pipeline, não sabe se ele está melhorando.

Métricas DORA

Métrica	O que mede
Deployment frequency	Frequência de deploys
Lead time for changes	Tempo do commit ao deploy
MTTR	Tempo para restaurar serviço
Change failure rate	% de deploys que falham

 Quatro métricas que separam times de elite dos demais.

Health checks pós-deploy

steps:

- run: `./deploy.sh production`
- run: `curl -f https://api.exemplo.com/health`
- run: `./verificar-version.sh`

- Valide **saúde** após o deploy
- Compare versão e métricas-chave
- Falha no health check → aciona rollback

⚠ Deploy sem verificação é um tiro no escuro.

Notificações de deploy

- **Slack/Teams:** sucesso e falha por canal
- **E-mail:** resumo diário ou deploys de produção
- **GitHub:** checks e status já nativos

 Notifique o time certo, no momento certo — sem spam.

Badges de status no README

```
![CI](https://github.com/SEU-USUARIO/SEU-REPO/actions/workflows/ci.yml/badge.svg)
```

- Mostra o status do pipeline publicamente
- Incentiva cultura de qualidade
- Disponível em **Actions** → **workflow** → **badge**

💡 Badge verde no README é marketing de engenharia.

Bloco 6

Estudo de Caso e Prática

Estudo de caso: cenário

Contexto

Aplicação web com **10 mil usuários** e deploys manuais aos sábados.

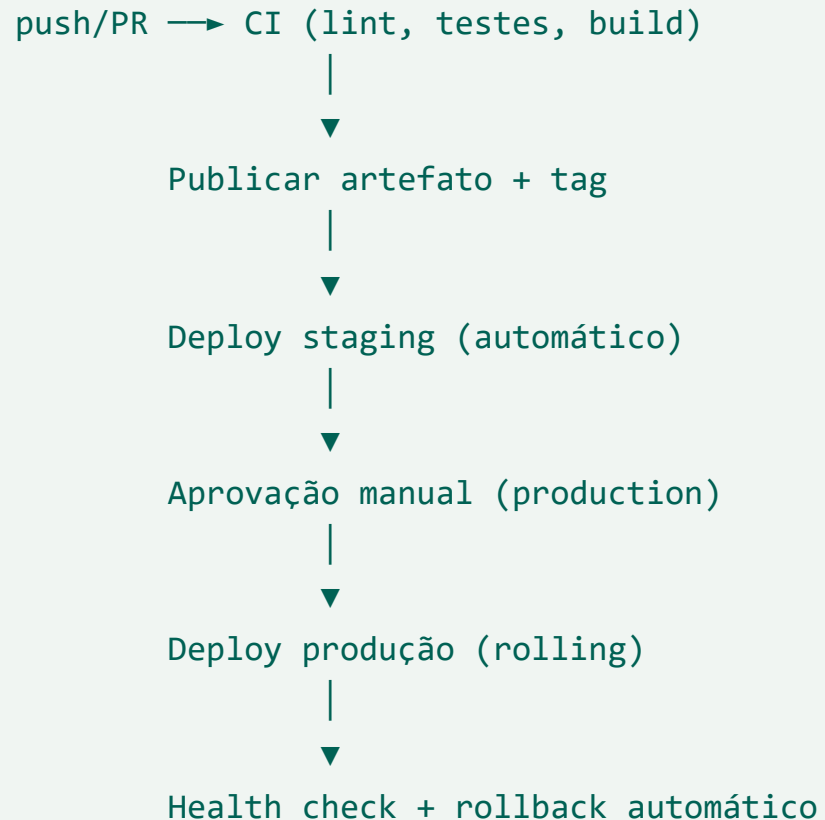
Problemas

- Deploy de 2h com downtime
- Rollback demorado e estressante
- Produção quebrada por mudança não testada

Objetivo

Entregar **diariamente** com risco controlado.

Estudo de caso: pipeline proposto



Resultado esperado: deploys diários, downtime zero, rollback em minutos.

Exercício prático (45 min)

Desafio

Monte um pipeline de CD completo:

1. Workflow de deploy com `environment: staging`
2. Aprovação manual para `production`
3. Deploy por `release` (tag SemVer)
4. Health check após o deploy
5. Concurrency para evitar deploys simultâneos

Reflexão

Qual estratégia de release você escolheria e por quê?

Checklist de um pipeline de CD

- ☐ Ambientes separados (staging/production)
- ☐ Aprovação manual em produção
- ☐ Secrets por ambiente, nunca no código
- ☐ Estratégia de release definida
- ☐ Rollback testado e documentado
- ☐ Health check pós-deploy
- ☐ Concurrency configurada
- ☐ Métricas DORA acompanhadas

O que aprendemos?

01

Ambientes e aprovações dão governança ao deploy

02

Estratégias de release equilibram risco e velocidade

03

Rollback e SemVer garantem o caminho de volta

04

Reutilização e concorrência tornam o pipeline sustentável

05

Segurança, observabilidade e DORA medem a maturidade



ESCOLA DO
FUTURO

Automação é cultura, não ferramenta.

CI/CD transforma entrega em rotina previsível.

Pratique: automatize um deploy esta semana.

Escola do Futuro de Goiás

Referências

- GitHub Docs — Environments: <https://docs.github.com/actions/deployment>
- GitHub Docs — Reusable workflows: <https://docs.github.com/actions/using-workflows/reusing-workflows>
- GitHub Docs — Security hardening: <https://docs.github.com/actions/security-guides/security-hardening-for-github-actions>
- DORA — Accelerate: State of DevOps
- Jez Humble & David Farley — Continuous Delivery
- Gene Kim — The Phoenix Project